



PYTHON · FOR ABSOLUTE BEGINNERS

# 파이썬 완전정복

1권 · 기초편

## 왕초보를 위한 가장 친절한 파이썬

프로그래밍을 난생처음 접하는 당신을 위해,  
한 걸음씩 쉽고 탄탄하게. 모든 예제는 실제로 실행해 검증했습니다.

비유로 이해하는 개념

실행 결과까지 확인

13개 장 · 미니 프로젝트

지은이 · Joon

# 목차

CONTENTS · 1권 기초편

01 파이썬을 시작하기 전에    프로그래밍이란? 설치와 첫 프로그램

02 변수와 자료형    이름표 상자, int·float·str·bool, 형변환

03 연산자    산술·비교·논리·할당과 우선순위

04 문자열 자유자재로    인덱싱·슬라이싱·메서드·f-string

05 입력과 출력    input(), print() 옵션, 형변환

06 조건문    if / elif / else 와 들여쓰기

07 반복문    for·while, break·continue, 구구단

08 자료구조    리스트·튜플·딕셔너리·셋

09 함수    def·return, 매개변수, 스코프

10 모듈과 표준 라이브러리    import, math·random·datetime

11 파일 입출력    open, with 문, 읽고 쓰기

12 예외 처리    try·except·finally, 에러 읽는 법

13 미니 프로젝트로 정리    계산기·할 일 목록·숫자 맞추기

# 파이썬을 시작하기 전에

컴퓨터에게 일을 시키는 방법, 그게 바로 프로그래밍이에요. 이 장에서는 프로그래밍이 뭔지, 왜 하필 파이썬인지 알아보고, 직접 설치해서 세상에서 가장 유명한 첫 프로그램을 만들어 봅니다. 겁먹지 마세요, 정말 쉬워요!

## 이 장에서 배울 것

- 프로그래밍과 프로그래밍 언어가 무엇인지
- 파이썬이 왜 초보자에게 좋은지, 어디에 쓰이는지
- 파이썬 설치와 두 가지 실행 방법(대화형 · 파일)
- 첫 프로그램 `print("Hello, World!")` 만들기

## 1.1 프로그래밍이란 무엇일까요?

**프로그래밍**은 한마디로 컴퓨터에게 시킬 일을 순서대로 적어 주는 것이에요. 요리 레시피를 떠올려 보세요. "물을 끓인다 → 면을 넣는다 → 3분 기다린다"처럼 순서대로 적힌 지시가 곧 레시피죠. 프로그래밍도 똑같아요. 컴퓨터가 알아들을 수 있는 말로 "이걸 해라, 그다음 저걸 해라" 하고 적는 거예요.

그런데 컴퓨터는 우리말이나 영어를 그대로 알아듣지 못해요. 그래서 컴퓨터와 대화하기 위한 특별한 언어가 필요한데, 그걸 **프로그래밍 언어**라고 불러요. 세상엔 수많은 프로그래밍 언어가 있는데, 우리가 배울 **파이썬(Python)**은 그중에서도 사람의 말과 가장 비슷하고 쉬운 언어랍니다.



### 이 책의 약속

어려운 용어가 나오면 반드시 쉬운 비유로 풀어 드려요. 그리고 모든 예제 코드는 저희가 실제로 컴퓨터에서 돌려 보고, 진짜 결과만 책에 실었어요. 그러니 안심하고 따라오세요.

## 1.2 왜 파이썬일까요?

수많은 언어 중에 왜 파이썬으로 시작할까요? 이유는 분명해요.

- **문법이 쉬워요** — 사람이 읽는 영어 문장과 비슷해서, 코드를 보면 무슨 뜻인지 감이 와요.
- **짧게 쓸 수 있어요** — 다른 언어로 열 줄 걸릴 일을 파이썬은 한두 줄로 끝내기도 해요.
- **어디에나 쓰여요** — 배워 두면 정말 다양한 분야에서 써먹을 수 있어요.
- **사용자가 아주 많아요** — 모르는 게 있을 때 검색하면 답이 넘쳐 나요.

## ▶ 파이썬은 어디에 쓰이나요?

| 분야       | 이런 걸 만들어요                  |
|----------|----------------------------|
| 웹 서비스    | 인스타그램, 유튜브 같은 서비스의 뒷단(서버)  |
| 데이터 분석   | 엑셀로는 벅찬 대용량 데이터를 정리하고 그래프로 |
| 인공지능(AI) | 챗봇, 이미지 인식, 추천 시스템         |
| 업무 자동화   | 반복되는 클릭·복붙 작업을 자동으로 처리     |
| 게임 · 교육  | 간단한 게임, 프로그래밍 교육           |

## 1.3 파이썬 설치하기

파이썬은 공식 홈페이지에서 무료로 내려받을 수 있어요. 주소는 [python.org](https://python.org) 예요.

1. 웹 브라우저에서 [python.org](https://python.org) 에 접속해요.
2. 상단 메뉴의 **Downloads** 를 눌러요. 내 컴퓨터에 맞는 버전(Windows / macOS)을 자동으로 추천해 줘요.
3. 노란색 **Download Python 3.x.x** 버튼을 눌러 설치 파일을 받아요.
4. 받은 파일을 실행하고 안내에 따라 설치해요.



### 윈도우 사용자 꼭 확인!

설치 첫 화면 맨 아래의 **Add Python to PATH** (또는 **Add python.exe to PATH**) 체크박스를 **반드시 켜고** 설치하세요. 이걸 놓치면 나중에 명령어가 인식되지 않아 애를 먹어요.



숫자 **3**으로 시작하는 버전(파이썬 3)을 받으면 돼요. 지금은 파이썬 3이 표준이라 특별히 신경 쓸 건 없어요. 설치가 끝나면 다음으로 넘어가요.

## 1.4 파이썬을 실행하는 두 가지 방법

파이썬을 쓰는 방법은 크게 두 가지예요. 상황에 맞게 골라 쓰면 돼요.

### ▶ ① 대화형 모드 — 한 줄씩 바로바로

**대화형(interactive) 모드**는 컴퓨터와 실시간으로 대화하듯 코드를 한 줄 입력하면 곧바로 결과를 보여 줘요. 계산기처럼 간단히 테스트할 때 좋아요. 터미널(윈도우는 명령 프롬프트)에서 **python3** (윈도우는 **python**)을 입력하면 **>>>** 라는 표시가 나와요. 이 표시가 "입력하세요"라는 신호예요.

대화형 모드 (>>> 는 파이썬이 주는 신호)

```
>>> 1 + 1
2
>>> print("안녕")
안녕
>>>
```

한 줄 입력할 때마다 즉시 답이 나오는 게 보이죠? 대화형 모드를 끝내려면 `exit()` 를 입력하면 돼요.

## ▶ ② 파일 모드 — 여러 줄을 저장했다가 한 번에

프로그램이 길어지면 매번 한 줄씩 치기 번거롭겠죠? 그럴 땐 코드를 **파일에 저장**해 두고 한 번에 실행해요. 파이썬 파일은 확장자가 `.py` 예요. 예를 들어 `hello.py` 라는 파일에 코드를 적어 두고, 터미널에서 이렇게 실행해요.

터미널에서 파일 실행

```
$ python3 hello.py
```



코드를 편하게 작성하려면 **코드 에디터**를 쓰면 좋아요. 초보자에게는 무료이고 인기 많은 **VS Code(비주얼 스튜디오 코드)**를 추천해요. 글자에 색이 입혀지고 오타도 잡아 줘서 훨씬 편하답니다. 없어도 메모장으로 시작할 수 있어요.

## 1.5 첫 프로그램: Hello, World!

이제 전 세계 프로그래머들이 처음 배울 때 만드는 바로 그 프로그램을 만들어 볼 거예요. 화면에 글자를 출력하는 `print()` 를 사용해요. `print` 는 영어로 '인쇄하다, 출력하다'라는 뜻이고, 괄호 ( ) 안에 화면에 보여 줄 내용을 넣어요.

파이썬 코드

```
print("Hello, World!")
```

실행 결과

```
Hello, World!
```

따옴표 " " 로 감싼 글자가 화면에 그대로 출력됐어요. 축하해요, 첫 프로그램 완성! 이번엔 한글도 출력하고, 여러 줄도 찍어 볼까요?

파이썬 코드

```
print("파이썬은 정말 쉬워요!")
print("한 줄 더 출력해볼까요?")
```

#### 실행 결과

파이썬은 정말 쉬워요!  
한 줄 더 출력해볼까요?

`print()` 를 두 번 쓰면 두 줄이 출력돼요. 위에서 아래로 순서대로 실행되는 게 보이죠?

이번엔 따옴표가 있을 때와 없을 때의 차이를 볼게요. 이게 앞으로 아주 중요해요.

#### 파이썬 코드

```
print(1 + 1)
print("1 + 1")
```

#### 실행 결과

2  
1 + 1



#### 따옴표가 핵심!

`print(1 + 1)` 은 따옴표가 없어서 파이썬이 **계산**을 해서 **2** 를 출력해요. 반면 `print("1 + 1")` 은 따옴표로 감싸서 **그냥 글자**로 보고 **1 + 1** 을 그대로 출력해요. 따옴표는 "이건 계산하지 말고 글자로 봐 줘"라는 신호예요.



#### 흔한 실수

괄호나 따옴표를 한쪽만 쓰면 오류가 나요. `print("안녕)` 처럼요. 여는 게 있으면 닫는 것도 꼭 짝을 맞춰 주세요. 또 따옴표는 큰따옴표 `"` 와 작은따옴표 `'` 둘 다 되지만, 여는 따옴표와 닫는 따옴표는 같은 종류여야 해요.

#### 연습문제

- 화면에 자신의 이름을 출력하는 프로그램을 만들어 보세요. (힌트: `print()` 안에 따옴표로 이름을 넣어요.)
- `print(10 * 5)` 와 `print("10 * 5")` 는 각각 무엇을 출력할까요? 먼저 예상해 보고 직접 실행해 확인해 보세요.
- `print()` 세 줄을 이용해 좋아하는 음식 세 가지를 한 줄에 하나씩 출력해 보세요.

#### 해답 · 힌트

② `print(10 * 5)` 는 계산해서 **50**, `print("10 * 5")` 는 글자 그대로 **10 \* 5** 를 출력해요. 따옴표의 유무가 계산이나 글자냐를 가른다는 걸 다시 확인할 수 있어요.

# 변수와 자료형

프로그램은 결국 '값'을 다루는 일이에요. 이름, 나이, 점수, 가격 같은 값들 말이죠. 이 값을 담아 두는 상자가 바로 '변수'예요. 그리고 값에는 여러 종류(자료형)가 있어요. 이 장에서 상자에 값을 담고 꺼내는 법, 값의 종류를 구별하고 서로 바꾸는 법을 배워요.

## 이 장에서 배울 것

- 변수가 무엇인지 (이름표 붙인 상자 비유)
- 네 가지 기본 자료형: 정수·실수·문자열·불
- `type()` 으로 자료형 확인하기
- 형변환: `int()`, `str()`, `float()`

## 2.1 변수 — 값을 담는 이름표 붙인 상자

변수(variable)는 값을 담아 두는 **상자**라고 생각하면 딱 맞아요. 그런데 상자가 여러 개면 헷갈리니까, 상자마다 **이름표**를 붙여요. 그 이름표가 바로 변수 이름이에요. 이름표를 보고 "아, 이 상자엔 나이가 들어 있구나" 하고 아는 거죠.

변수를 만드는 건 아주 간단해요. **이름 = 값** 형태로 쓰면 돼요. 여기서 **=** 는 "같다"는 뜻이 아니라 **"오른쪽 값을 왼쪽 상자에 넣어라"**는 뜻이에요. 이걸 '할당'이라고 불러요.

파이썬 코드

```
name = "지우"
age = 8
print(name)
print(age)
```

실행 결과

```
지우
8
```

`name` 이라는 상자에 **"지우"** 를, `age` 라는 상자에 **8** 을 넣었어요. 그다음 `print(name)` 으로 상자 안의 값을 꺼내 출력했죠. 변수 이름을 쓰면 그 안에 담긴 값이 튀어나온다고 생각하면 돼요.

### ▶ 상자의 값은 언제든지 바꿀 수 있어요

변수는 말 그대로 'variable(변할 수 있는)'이에요. 같은 상자에 새 값을 넣으면 이전 값은 사라지고 새 값으로 바뀌어요.

파이썬 코드

```
box = "사과"
print(box)
box = "바나나"
print(box)
```

실행 결과

```
사과
바나나
```



### 변수 이름 짓기 규칙

① 영문자·숫자·밑줄(\_)로 짓고, ② 숫자로 시작하면 안 돼요, ③ 띄어쓰기는 안 되니 `my_age` 처럼 밑줄로 이어요. 무엇보다 **값이 뭔지 알 수 있는 이름이 좋아요.** `a` 보다 `age` 가 나중에 코드를 읽을 때 훨씬 편하답니다.

## 2.2 자료형 — 값에도 종류가 있어요

상자에 담는 값에는 여러 **종류(자료형, type)**가 있어요. 숫자냐 글자냐에 따라 컴퓨터가 다르게 다뤄요. 기초편에서 꼭 알아야 할 네 가지를 볼게요.

| 자료형 | 영어 이름 | 무엇을 담나          | 예시          |
|-----|-------|-----------------|-------------|
| 정수  | int   | 소수점 없는 숫자       | 10, -3, 0   |
| 실수  | float | 소수점 있는 숫자       | 3.14, -0.5  |
| 문자열 | str   | 글자·문장 (따옴표로 감쌌) | "안녕", "a"   |
| 불   | bool  | 참/거짓 둘 중 하나     | True, False |

각 값의 자료형이 궁금할 땐 `type()` 을 써요. 괄호 안에 값이나 변수를 넣으면 그 종류를 알려 줘요.

파이썬 코드

```
a = 10          # 정수(int)
b = 3.14        # 실수(float)
c = "안녕"      # 문자열(str)
d = True        # 불(bool)

print(type(a))
print(type(b))
print(type(c))
print(type(d))
```



실행 결과

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

`<class 'int'>` 는 "이 값은 int(정수) 종류야"라는 뜻이에요. `class` 라는 단어는 지금은 '종류'라는 뜻으로만 받아들이면 충분해요.



#### 불(bool)은 참·거짓

`True` (참)와 `False` (거짓), 딱 두 값만 가져요. 첫 글자가 **대문자**인 점에 주의하세요. `true` 라고 쓰면 오류가 나요. 불은 6장 조건문에서 아주 유용하게 쓰여요.



#### 숫자인데 문자열인 경우?

`10` 은 정수지만, `"10"` 처럼 따옴표로 감싸면 문자열이에요! 겉보기엔 같은 10이지만 컴퓨터엔 전혀 다른 값이에요. 문자열 `"10"` 으로는 계산을 할 수 없어요. 이게 다음에 배울 '형변환'이 필요한 이유예요.

## 2.3 형변환 — 자료형을 서로 바꾸기

값의 종류를 바꾸는 걸 **형변환(type casting)**이라고 해요. 예를 들어 문자열 `"100"` 을 진짜 숫자 `100` 으로 바꾸면 계산에 쓸 수 있죠. 바꾸는 방법은 자료형 이름을 함수처럼 쓰는 거예요.

### ▶ 문자열 → 정수: `int()`

파이썬 코드

```
num_str = "100"
num_int = int(num_str)
print(num_int + 50)
```

실행 결과

150

`"100"` 은 문자열이라 그대로는 `+ 50` 을 못 해요. `int()` 로 정수 `100` 으로 바꾸니 계산이 되어 `150` 이 나왔죠. 5장에서 사용자가 입력한 값을 다룰 때 이 기술을 아주 많이 써요.

### ▶ 정수 → 문자열: str()

파이썬 코드

```
age = 8
message = "제 나이는 " + str(age) + "살입니다."
print(message)
```

실행 결과

```
제 나이는 8살입니다.
```

문자열끼리는 `+` 로 이어 붙일 수 있어요. 하지만 문자열과 숫자는 바로 못 붙여요. 그래서 숫자 `age` 를 `str()` 로 문자열로 바꾼 뒤 이어 붙였어요.

### ▶ 문자열 → 실수: float()

파이썬 코드

```
pi = float("3.14")
print(pi)
print(type(pi))
```

실행 결과

```
3.14
<class 'float'>
```

### ▶ 실수 → 정수: int() (소수점 아래는 버려요)

파이썬 코드

```
x = int(3.9)
print(x)
```

실행 결과

```
3
```

**⚠ 실수를 정수로 바꾸면 반올림이 아니에요!**

`int(3.9)` 는 4 가 아니라 3 이에요. 반올림하지 않고 소수점 아래를 그냥 버려요. 반올림이 필요하다면 나중에 배울 `round()` 를 쓰면 돼요.



문자열이 숫자 모양이 아니면 `int()` 가 오류를 내요. 예를 들어 `int("사과")` 는 바꿀 숫자가 없어서 에러가 나요. 이런 오류를 다루는 법은 12장 예외 처리에서 배워요.

## 연습문제

1. `height = 175.5` 라는 변수를 만들고, `type()` 으로 자료형을 출력해 보세요. 무엇이 나올까요?
2. 문자열 `"20"` 과 `"30"` 을 각각 정수로 바꿔 더한 결과를 출력해 보세요. (힌트: `int()` )
3. 자신의 나이를 담은 정수 변수를 만들고, `"저는 00살이에요"` 형태의 문장을 `+` 로 이어 출력해 보세요. (힌트: `str()` 필요)

---

### 해답 · 힌트

- ① `<class 'float'>` 가 나와요(소수점이 있으니 실수). ② `int("20") + int("30")` → `50`. ③ 예: `age = 8` 다음에 `print("저는 " + str(age) + "살이에요")`.

# 연산자

컴퓨터는 원래 계산하는 기계였어요. 그래서 파이썬은 계산을 아주 잘합니다. 더하고 빼는 산술 계산부터, 두 값을 비교하고, 여러 조건을 묶는 논리 연산까지. 이 장에서 배우는 연산자들은 앞으로 조건문·반복문에서 계속 등장하니 꼭 익혀 두세요.

## 이 장에서 배울 것

- 산술 연산자: `+` `-` `*` `/` `//` `%` `**`
- 비교 연산자: `==` `!=` `<` `>` 와 참·거짓
- 논리 연산자: `and` , `or` , `not`
- 할당 연산자 `+=` 와 연산 우선순위

## 3.1 산술 연산자 — 사칙연산과 그 친구들

먼저 우리가 익숙한 더하기·빼기·곱하기·나누기예요. 곱하기는 `x` 대신 별표 `*`, 나누기는 `÷` 대신 슬래시 `/` 를 써요.

파이썬 코드

```
print(7 + 3)
print(7 - 3)
print(7 * 3)
print(7 / 3)
```

실행 결과

```
10
4
21
2.3333333333333335
```

여기서 하나 눈여겨볼 점! `7 / 3` 의 결과가 `2.333...` 이죠? 파이썬에서 `/` 나눗셈은 항상 실수(소수점)로 답을 줘요. `6 / 2` 도 `3` 이 아니라 `3.0` 이 나온답니다.

### ▶ 몫(`//`), 나머지(`%`), 거듭제곱(`**`)

초보자가 처음 보면 낯선 세 가지예요. 하나씩 볼게요.

파이썬 코드

```
print(7 // 3) # 몫
print(7 % 3)  # 나머지
print(2 ** 10) # 거듭제곱
```

실행 결과

```
2
1
1024
```

- `//` (몫): `7 // 3` → 7을 3으로 나눈 몫은 `2`.
- `%` (나머지): `7 % 3` → 7을 3으로 나눈 나머지는 `1`. `%` 는 '어떤 수로 나누어떨어지는지' 판단할 때 자주 써요(짝수/홀수 판별 등).
- `**` (거듭제곱): `2 ** 10` → 2를 10번 곱한 값 `1024`.



#### 나머지 `%` 의 활용

어떤 수가 짝수인지 알고 싶으면 `2` 로 나눈 나머지를 봐요. 나머지가 `0` 이면 짝수예요. 예: `10 % 2` 는 `0` (짝수), `7 % 2` 는 `1` (홀수). 이 기법은 6장에서 다시 만나요.

## 3.2 비교 연산자 — 두 값을 견주다

두 값을 비교해서 참(True)인지 거짓(False)인지 알려 주는 연산자예요. 결과는 항상 불(bool) 값이에요.

| 연산자                | 의미     | 예시                     | 결과                 |
|--------------------|--------|------------------------|--------------------|
| <code>==</code>    | 같다     | <code>5 == 5</code>    | <code>True</code>  |
| <code>!=</code>    | 다르다    | <code>5 != 3</code>    | <code>True</code>  |
| <code>&lt;</code>  | 작다     | <code>3 &lt; 5</code>  | <code>True</code>  |
| <code>&gt;</code>  | 크다     | <code>3 &gt; 5</code>  | <code>False</code> |
| <code>&lt;=</code> | 작거나 같다 | <code>5 &lt;= 5</code> | <code>True</code>  |
| <code>&gt;=</code> | 크거나 같다 | <code>3 &gt;= 5</code> | <code>False</code> |

파이썬 코드

```
print(5 == 5)
print(5 != 3)
print(3 < 5)
print(10 >= 20)
```

실행 결과

```
True
True
True
False
```



**= 와 == 는 완전히 달라요!**

= (하나)는 값을 담는 할당이고, == (둘)는 두 값이 같은지 비교하는 거예요. "a는 5다"( `a = 5` )와 "a가 5랑 같니?"( `a == 5` )의 차이죠. 초보자가 정말 많이 헷갈리는 부분이니 꼭 구분하세요.

### 3.3 논리 연산자 — 여러 조건을 묶기

여러 개의 참·거짓을 묶어서 판단할 때 써요. 세 가지가 있어요.

- `and` (그리고): 둘 다 참일 때만 참.
- `or` (또는): 하나라도 참이면 참.
- `not` (아니다): 참↔거짓을 뒤집어요.

파이썬 코드

```
print(True and False)
print(True or False)
print(not True)
```

실행 결과

```
False
True
False
```

실제로는 이렇게 조건을 조합해서 써요. 예를 들어 '점수가 60 이상이면서 100 이하'인지 확인해 볼게요.

파이썬 코드

```
score = 85
print(score >= 60 and score <= 100)
```

실행 결과

```
True
```

`score >= 60` (참)이고 `score <= 100` (참)이라, 둘 다 참이니까 `and` 의 결과는 `True` 예요. 이런 조합은 6장 조건문에서 진가를 발휘해요.

### 3.4 할당 연산자 — 값을 갱신하는 지름길

변수에 든 값을 바꿀 때 자주 쓰는 편리한 표현이에요. `money = money + 500` 을 짧게 `money += 500` 이라고 쓸 수 있어요.

파이썬 코드

```
money = 1000
money += 500    # money = money + 500
print(money)
money -= 200
print(money)
money *= 2
print(money)
```

실행 결과

```
1500
1300
2600
```

| 표현                  | 같은 뜻                   | 의미           |
|---------------------|------------------------|--------------|
| <code>a += 3</code> | <code>a = a + 3</code> | 3을 더해 다시 담기  |
| <code>a -= 3</code> | <code>a = a - 3</code> | 3을 빼서 다시 담기  |
| <code>a *= 3</code> | <code>a = a * 3</code> | 3을 곱해 다시 담기  |
| <code>a /= 3</code> | <code>a = a / 3</code> | 3으로 나눠 다시 담기 |



`+=` 는 7장 반복문에서 '숫자를 하나씩 세거나 합계를 누적'할 때 정말 많이 써요. 예: `total += 점수` 를 반복하면 점수가 차곡차곡 더해져요.

### 3.5 연산 우선순위 — 무엇을 먼저 계산할까

수학에서 곱셈·나눗셈을 덧셈·뺄셈보다 먼저 하는 것처럼, 파이썬에도 **계산 순서**가 있어요. 순서를 바꾸고 싶으면 **괄호**로 묶으면 돼요. 괄호 안이 가장 먼저 계산돼요.

파이썬 코드

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

#### 실행 결과

14

20

$2 + 3 * 4$  는 곱셈을 먼저 해서  $2 + 12 = 14$  ,  $(2 + 3) * 4$  는 괄호를 먼저 해서  $5 * 4 = 20$  이에요. 헷갈릴 것 같으면 **괄호를 넉넉히 쓰는 습관**이 좋아요. 실수도 줄고 읽기도 편해요.

#### 연습문제

1. 17을 5로 나눈 몫과 나머지를 각각 출력해 보세요. (힌트: `//` 와 `%` )
2. `15 % 2` 의 결과로 15가 짝수인지 홀수인지 판단해 보세요. 결과가 무엇인가요?
3. 변수 `coin = 100` 을 만든 뒤, `+=` 로 50을 더하고, 다시 `*=` 로 2배 한 결과를 출력해 보세요.

#### 해답 · 힌트

① 몫 `3` , 나머지 `2` . ② `15 % 2` 는 `1` → 나머지가 0이 아니므로 홀수. ③ `100` → `150` → `300` .



# 문자열 자유자재로

프로그램에서 다루는 값의 절반은 '글자'라고 해도 과언이 아니에요. 이름, 주소, 메시지, 파일 내용... 모두 문자열이죠. 파이썬은 문자열을 아주 편하게 다룰 수 있는 도구를 잔뜩 준비해 뒀어요. 이 장에서 글자를 자르고, 바꾸고, 예쁘게 조립하는 법을 배워요.

## 이 장에서 배울 것

- 따옴표로 문자열 만들기, 인덱싱과 슬라이싱
- 유용한 메서드: `upper·lower·strip·split·replace·find`
- f-string 으로 값 끼워 넣기
- 이스케이프 문자 `\n`, `\"`

## 4.1 문자열 만들기과 인덱싱

문자열은 따옴표로 감싼 글자예요. 큰따옴표 `"..."` 든 작은따옴표 `'...'` 든 돼요. 문자열의 각 글자에는 **번호(인덱스)**가 매겨져 있어요. 중요한 건 이 번호가 **0부터 시작**한다는 점이에요! 첫 글자가 0번, 두 번째가 1번...

글자를 하나 꺼낼 땐 `변수[번호]` 처럼 대괄호에 번호를 넣어요. 음수를 넣으면 **뒤에서부터** 세요. `-1` 은 맨 마지막 글자예요.

파이썬 코드

```
word = "Python"
print(word[0])
print(word[1])
print(word[-1])
```

실행 결과

```
P
y
n
```

`P-y-t-h-o-n` 에서 0번은 `P`, 1번은 `y`, 뒤에서 첫 번째(-1)는 `n` 이예요.



### 번호는 0부터!

프로그래밍에서는 순서를 셀 때 대부분 0부터 세요. 처음엔 어색하지만 금방 익숙해져요. '첫 번째'가 0번이라는 것만 기억하세요. 이 규칙은 8장 리스트에서도 똑같이 적용돼요.

## 4.2 슬라이싱 — 문자열을 잘라 내기

여러 글자를 한 번에 잘라 내는 걸 **슬라이싱(slicing)**이라고 해요. 빵을 슬라이스로 자르는 그 느낌이에요. **변수 [시작:끝]** 형태로 쓰는데, 끝 번호는 포함하지 않아요. 즉 **[0:3]** 은 0, 1, 2번까지예요.

파이썬 코드

```
word = "Python"
print(word[0:3])
print(word[3:])
print(word[:3])
print(word[::-2])
```

실행 결과

```
Pyt
hon
Pyt
Pto
```

- **word[0:3]** → 0,1,2번 = **Pyt** (끝 3은 포함 안 함).
- **word[3:]** → 3번부터 끝까지 = **hon** (끝을 비우면 '끝까지').
- **word[:3]** → 처음부터 3번 전까지 = **Pyt** (시작을 비우면 '처음부터').
- **word[::-2]** → 처음부터 끝까지 **2칸씩 건너뛰며** = **Pto**.



슬라이싱의 세 번째 값( **[::-2]** 의 2)은 '몇 칸씩 건너뛰지'예요. **[::-1]** 처럼 -1을 넣으면 문자열을 거꾸로 뒤집을 수도 있어요. 재미있죠?

## 4.3 문자열 메서드 — 글자를 다루는 도구들

문자열에는 **메서드(method)**라는 편리한 기능들이 달려 있어요. 메서드는 **문자열.기능()** 형태로 점( . )을 찍어 사용해요. '이 글자에게 이 일을 시켜라'라고 명령하는 느낌이에요.

### ▶ 대소문자 변환과 공백 제거: upper / lower / strip

파이썬 코드

```
text = " Hello Python "
```

```
print(text.upper())
print(text.lower())
print(text.strip())
```

#### 실행 결과

```
HELLO PYTHON
hello python
Hello Python
```

- `.upper()` — 모두 대문자로.
- `.lower()` — 모두 소문자로.
- `.strip()` — 양 끝의 공백(빈칸)을 제거. 사용자 입력을 정리할 때 유용해요.

### ▶ 나누기와 바꾸기: `split` / `replace`

#### 파이썬 코드

```
sentence = "사과,바나나,포도"
fruits = sentence.split(",")
print(fruits)
```

#### 실행 결과

```
['사과', '바나나', '포도']
```

`.split(",")` 는 쉼표를 기준으로 문자열을 잘라 **리스트**로 만들어요. (리스트는 8장에서 자세히 배워요. 지금은 '여러 값을 담은 묶음' 정도로 알면 돼요.)

#### 파이썬 코드

```
msg = "나는 자바를 좋아해"
print(msg.replace("자바", "파이썬"))
```

#### 실행 결과

```
나는 파이썬을 좋아해
```

`.replace("자바", "파이썬")` 는 문자열 안의 '자바'를 모두 '파이썬'으로 바꿔요.

### ▶ 찾기: `find`

#### 파이썬 코드

```
word = "banana"
print(word.find("a"))
print(word.find("z"))
```

실행 결과

1  
-1

`.find("a")` 는 'a'가 처음 등장하는 위치(번호)를 알려 줘요. `banana` 의 첫 'a'는 1번이죠. 찾는 글자가 없으면 `-1` 을 돌려 줘요.



**메서드는 원본을 바꾸지 않아요**

`text.upper()` 를 해도 원래 `text` 는 그대로예요. 메서드는 **바뀐 결과를 새로 돌려줄 뿐**이에요. 바뀐 값을 계속 쓰려면 `text = text.upper()` 처럼 다시 담아야 해요.

## 4.4 f-string — 문자열에 값 끼워 넣기

2장에서 `"제 나이는 " + str(age) + "살"` 처럼 `+` 로 이어 붙였던 거 기억나죠? 조금 번거로웠어요. 이것 훨씬 깔끔하게 하는 방법이 **f-string**이에요. 따옴표 앞에 `f` 를 붙이고, 값을 넣고 싶은 곳에 `{변수}` 를 쓰면 끝이에요.

파이썬 코드

```
name = "지우"
age = 8
print(f"제 이름은 {name}이고, 나이는 {age}살이에요.")
```

실행 결과

제 이름은 지우이고, 나이는 8살이에요.

중괄호 `{ }` 안의 변수 값이 자동으로 그 자리에 들어가요. `str()` 같은 형변환도 알아서 해 주니 정말 편해요. 심지어 중괄호 안에서 **계산**도 할 수 있어요!

파이썬 코드

```
price = 3000
count = 4
print(f"총 금액: {price * count}원")
```

실행 결과

총 금액: 12000원



f-string은 파이썬에서 가장 자주 쓰는 문자열 기법이에요. 앞으로 이 책의 예제에도 계속 등장하니 여기서 확실히 익혀 두세요. `f` 를 빠뜨리면 `{name}` 이 글자 그대로 출력되니 주의!

## 4.5 이스케이프 문자 — 특별한 글자 표현

줄바꿈이나 따옴표처럼 그냥은 넣기 어려운 글자는 백슬래시 `\` 를 이용한 **이스케이프 문자**로 표현해요.

파이썬 코드

```
print("첫째 줄\n둘째 줄")
print("따옴표 안의 \"큰따옴표\"")
```

실행 결과

```
첫째 줄
둘째 줄
따옴표 안의 "큰따옴표"
```

| 표현              | 의미          |
|-----------------|-------------|
| <code>\n</code> | 줄바꿈(다음 줄로)  |
| <code>\t</code> | 탭(간격 띄우기)   |
| <code>\"</code> | 큰따옴표 글자 그대로 |
| <code>\\</code> | 백슬래시 글자 그대로 |

`\n` 은 화면엔 안 보이지만 '여기서 줄을 바꿔라'라는 신호예요. 큰따옴표로 감싼 문자열 안에 큰따옴표를 넣고 싶으면 `\"` 로 써서 "이건 문자열을 끝내는 게 아니라 그냥 따옴표 글자야"라고 알려 줘요.

### 연습문제

- `city = "Seoul"` 에서 첫 글자와 마지막 글자를 각각 인덱싱으로 출력해 보세요.
- 전화번호 문자열 `"010-1234-5678"` 을 `split("-")` 로 나눠 출력해 보세요. 결과는 어떤 모양일까요?
- 변수 `name` 과 `score` 를 만들고, f-string으로 `"00님의 점수는 00점입니다"` 형태로 출력해 보세요.

#### 해답 · 힌트

① 첫 글자 `s (= city[0])`, 마지막 `l (= city[-1])`. ② `['010', '1234', '5678']` — 세 조각의 리스트. ③ 예: `print(f"{name}님의 점수는 {score}점입니다")`.

# 입력과 출력

지금까지는 우리가 코드에 값을 직접 적어 넣었어요. 하지만 진짜 프로그램은 사용자와 '대화'해요. 이름을 물어보고, 숫자를 받아 계산해 주죠. 이 장에서는 사용자에게 값을 입력받는 법과, 결과를 원하는 모양으로 예쁘게 출력하는 법을 배워요.

## 이 장에서 배울 것

- `input()` 으로 사용자 입력받기
- 입력값은 항상 문자열 — 형변환의 필요성
- `print()` 의 옵션: `sep`, `end`
- 입력받아 계산하는 실전 흐름

## 5.1 input() — 사용자에게 물어보기

`input()` 은 사용자가 키보드로 무언가 입력할 때까지 기다렸다가, 입력한 값을 돌려주는 함수예요. 괄호 안에 안내 문구를 넣으면 화면에 그 문구를 보여 줘요.

● ● ● 파이썬 코드

```
name = input("이름을 입력하세요: ")
print("안녕하세요, " + name + "님!")
```

사용자가 `지우` 라고 입력하고 엔터를 치면, 그 값이 `name` 상자에 담겨요. 실제로 실행하면 이렇게 흘러가요(입력값을 `지우` 라고 가정).

실행 예시 (밑줄 친 부분이 사용자 입력)

```
이름을 입력하세요: 지우
안녕하세요, 지우님!
```



이 책의 코드 예제는 미리 실행해 검증하지만, `input()` 은 사람이 직접 입력해야 하는 부분이에요. 그래서 이 장의 예제는 '이렇게 입력하면 이런 결과'를 보여 드리는 방식으로 설명해요. 여러분은 직접 값을 입력해 보며 확인하면 돼요.

## 5.2 input()의 함정 — 입력값은 무조건 문자열!

여기 초보자가 반드시 알아야 할 중요한 사실이 있어요. `input()` 이 돌려주는 값은 언제나 문자열(str)이에요. 사용자가 숫자 `8` 을 입력해도, 파이썬은 그걸 문자열 `"8"` 로 받아요. 그래서 계산을 하려면 2장에서 배운 `int()` 형변환이 꼭 필요해요.

파이썬 코드

```
age = input("나이를 입력하세요: ")
age = int(age) # 문자열을 정수로 바꿔야 계산 가능
print("내년이면", age + 1, "살이 되네요!")
```

사용자가 8 을 입력했다고 가정하면 결과는 이래요.

실행 결과

내년이면 9 살이 되네요!



#### 형변환을 빠뜨리면?

`age = input(...)` 뒤에 `int()` 없이 바로 `age + 1` 을 하면, 문자열 "8" 에 숫자 1 을 더하려다 오류가 나요. 사용자에게 숫자를 받으면 곧바로 `int()` 로 바꾼다고 습관을 들이세요.

입력과 형변환을 한 줄로 합칠 수도 있어요. 자주 쓰는 형태예요.

파이썬 코드

```
age = int(input("나이를 입력하세요: "))
```

`input()` 이 받은 문자열을 곧바로 `int()` 로 감싸 정수로 만드는 거예요. 안쪽부터 실행된다고 생각하면 이해가 쉬워요: 먼저 입력받고 → 그다음 정수로 변환.

## 5.3 print()를 더 똑똑하게 — sep과 end

`print()` 는 값 여러 개를 쉼표로 나열해 한 번에 출력할 수 있어요. 이때 값들 사이는 기본적으로 빈칸 하나로 구분돼요. 이 구분자를 바꾸는 게 `sep` 옵션이에요.

파이썬 코드

```
print("사과", "바나나", "포도")
print("사과", "바나나", "포도", sep=", ")
print("사과", "바나나", "포도", sep=" / ")
```

실행 결과

사과 바나나 포도  
사과, 바나나, 포도  
사과 / 바나나 / 포도

`sep` (separator, 구분자)에 원하는 문자를 주면 값들 사이에 그게 들어가요. 표나 목록을 예쁘게 출력할 때 유용해요.

## ▶ end — 줄바꿈을 바꾸기

`print()` 는 출력이 끝나면 자동으로 줄을 바꿔요(다음 `print` 는 아랫줄에 찍힘). 이 '끝맺음'을 바꾸는 게 `end` 옵션이에요. `end=""` 로 주면 줄을 바꾸지 않아 여러 `print` 가 한 줄에 이어져요.

파이썬 코드

```
print("로딩 중", end="")
print("...", end="")
print("완료!")
```

실행 결과

로딩 중...완료!



`sep` 과 `end` 는 이름을 꼭 적어서( `sep=` , `end=` ) 줘야 해요. 이렇게 이름을 붙여 주는 값을 '키워드 인자'라고 하는데, 9장 함수에서 자세히 배워요. 지금은 '옵션을 이렇게 준다' 정도로 기억하면 충분해요.

## 5.4 실전: 입력받아 계산하기

배운 걸 합쳐서 '두 숫자를 입력받아 더하는' 작은 프로그램을 만들어 볼게요. 실제 프로그램의 기본 뼈대예요: **입력 → 처리 → 출력**.

파이썬 코드

```
a = int(input("첫 번째 숫자: "))
b = int(input("두 번째 숫자: "))
print("합은", a + b, "입니다.")
```

사용자가 각각 3 과 5 를 입력했다고 하면 결과는 이래요.

실행 결과

합은 8 입니다.

입력을 문자열로 받아 `int()` 로 숫자로 바꾸고, 더한 뒤 출력했어요. 이 '입력 → 처리 → 출력' 흐름은 앞으로 만들 거의 모든 프로그램의 기본 구조입니다.



## 연습문제

1. 사용자에게 좋아하는 색을 입력받아 "00색이 참 예쁘네요!" 라고 출력하는 프로그램을 만들어 보세요.
2. 두 숫자를 입력받아 곱한 결과를 출력해 보세요. (힌트: `int()` 두 번, `*`)
3. `print("가", "나", "다", sep="-")` 의 출력을 예상한 뒤 실행해 확인해 보세요.

---

### 해답 · 힌트

① 예: `color = input("좋아하는 색: ")` 다음 `print(f"{color}색이 참 예쁘네요!")`. ③ 결과는 가-나-다 — 값들 사이에 - 가 들어가요.

# 조건문

'만약 비가 오면 우산을 챙긴다.' 우리는 늘 상황에 따라 다르게 행동해요. 프로그램도 마찬가지예요. 조건에 따라 다른 일을 하도록 만드는 게 바로 조건문이에요. 여기서부터 프로그램이 '똑똑하게' 판단하기 시작해요. 3장에서 배운 비교·논리 연산자가 크게 활약합니다.

## 이 장에서 배울 것

- `if` 로 조건에 따라 실행하기
- `if` · `elif` · `else` 로 여러 갈래 나누기
- 들여쓰기가 왜 그렇게 중요한지
- 중첩 조건과 실전 예제(성적 판정)

## 6.1 if — 만약 ~라면

`if`는 영어로 '만약'이에요. `if` 뒤에 조건을 쓰고 콜론 `:` 을 붙인 다음, 조건이 참(True)일 때 실행할 코드를 들여쓰기해서 적어요.

● ● ● 파이썬 코드

```
age = 20
if age >= 19:
    print("성인입니다.")
```

실행 결과

성인입니다.

`age >= 19` 가 참(20은 19 이상)이라서 안쪽의 `print` 가 실행됐어요. 만약 `age` 가 15였다면 조건이 거짓이라 아무것도 출력되지 않았을 거예요.



### 들여쓰기 = 소속 표시

`if` 아래에서 들여쓰기된 줄들이 '조건이 참일 때 실행할 코드'예요. 들여쓰기는 보통 공백 4칸(스페이스바 4번)으로 해요. 파이썬은 이 들여쓰기로 코드의 소속을 판단하기 때문에, 다른 언어와 달리 들여쓰기가 문법 그 자체예요. 아주 중요해요!



### 콜론(:)과 들여쓰기를 빠뜨리지 마세요

`if` 줄 끝에 `:` 을 안 붙이거나, 다음 줄을 들여쓰지 않으면 오류가 나요. 또 들여쓰기 칸 수가 들쭉날쭉하면 `IndentationError` 라는 에러가 나니 일정하게 4칸으로 맞추세요.

## 6.2 else — 그렇지 않으면

조건이 거짓일 때 할 일을 정하고 싶으면 `else` (그렇지 않으면)를 써요. `if` 가 참이면 `if` 쪽을, 거짓이면 `else` 쪽을 실행해요. 둘 중 하나만!

파이썬 코드

```
temp = 15
if temp >= 25:
    print("덥네요, 반팔을 입어요.")
else:
    print("쌀쌀하니 겉옷을 챙겨요.")
```

실행 결과

쌀쌀하니 겉옷을 챙겨요.

`temp` 가 15라 `temp >= 25` 는 거짓이에요. 그래서 `else` 쪽이 실행됐죠. `else` 에는 조건을 쓰지 않아요(그냥 '나머지 전부'니까요).

## 6.3 elif — 여러 갈래로 나누기

갈림길이 셋 이상이면 `elif`(else if의 줄임말, '아니고 만약')를 써요. `if` → `elif` → `elif` → ... → `else` 순서로 위에서부터 조건을 검사하다가, 처음으로 참이 되는 곳만 실행하고 나머지는 건너뛰어요.

파이썬 코드

```
score = 85
if score >= 90:
    print("A 학점")
elif score >= 80:
    print("B 학점")
elif score >= 70:
    print("C 학점")
else:
    print("재시험")
```

실행 결과

B 학점

`score` 가 85예요. 첫 조건 `>= 90` 은 거짓, 다음 `>= 80` 은 참이라 여기서 `B 학점` 을 출력하고 멈춰요. 아래 조건들은 검사조차 하지 않아요. 위에서부터 순서대로, 처음 참인 곳에서 끝이라는 흐름을 꼭 기억하세요.



#### 조건 순서가 중요해요

위 예제에서 조건을 큰 수부터 배치한 이유가 있어요. 만약 `>= 70` 을 맨 위에 두면 85점도 70 이상이라 무조건 'C 학점'이 되어 버려요. `elif` 는 위에서부터 걸리니, 더 좁은(엄격한) 조건을 위에 두어야 해요.

## 6.4 중첩 조건 — 조건 안의 조건

조건문 안에 또 조건문을 넣을 수도 있어요. 들여쓰기를 한 단계 더 하면 돼요. 예를 들어 '돈이 충분한지' 확인한 뒤, 그 안에서 '회원인지'를 또 따져 볼 수 있어요.

파이썬 코드

```

money = 5000
is_member = True
if money >= 3000:
    if is_member:
        print("회원가로 구매 완료!")
    else:
        print("일반가로 구매 완료!")
else:
    print("잔액이 부족해요.")

```

실행 결과

회원가로 구매 완료!

바깥 `if` (돈 충분?)가 참이라 안으로 들어가고, 안쪽 `if` (회원?)도 참이라 '회원가로 구매 완료!'가 출력됐어요. 들여쓰기 단계로 소속을 구분하는 게 보이죠? 안쪽 코드는 8칸(두 단계) 들여쓰기됐어요.



중첩이 너무 깊어지면 코드가 읽기 어려워져요. 3장에서 배운 `and` 를 쓰면 `if money >= 3000 and is_member:` 처럼 한 줄로 합칠 수도 있어요. 상황에 맞게 더 읽기 쉬운 쪽을 고르세요.

## 6.5 실전: 훌쩍 판별기

3장에서 배운 나머지 연산 `%` 와 조건문을 합치면 숫자가 양수·음수·0 중 무엇인지 판별할 수 있어요.

#### 파이썬 코드

```
num = -4
if num > 0:
    print("양수")
elif num == 0:
    print("영")
else:
    print("음수")
```

#### 실행 결과

음수

`num` 이 -4라 첫 조건도 둘째 조건도 거짓이고, 최종적으로 `else` 의 '음수'가 출력됐어요. 이렇게 조건문은 상황을 여러 경우로 나눠 알맞은 행동을 고르게 해 줘요.

#### 연습문제

1. 나이를 입력받아 19세 이상이면 "입장 가능", 아니면 "입장 불가" 를 출력해 보세요.
2. 점수를 입력받아 60점 이상이면 "합격", 아니면 "불합격" 을 출력해 보세요.
3. 숫자를 입력받아 짝수면 "짝수", 홀수면 "홀수" 를 출력해 보세요. (힌트: `num % 2 == 0`)

#### 해답 · 힌트

③ `num % 2` 가 0 이면 짝수예요. `if num % 2 == 0:` → "짝수", `else:` → "홀수". 입력은 `num = int(input("숫자: "))` 로 받으면 돼요.

# 반복문

'구구단 2단을 9번 출력해라.' 같은 코드를 아홉 번 복붙할 순 없겠죠? 똑같은 일을 여러 번 반복할 때 쓰는 게 바로 반복문이에요. 컴퓨터가 가장 잘하는 게 반복이거든요. 이 장에서 `for` 와 `while`, 그리고 반복을 제어하는 기술을 배우면 코드가 훨씬 강력해져요.

## 이 장에서 배울 것

- `for` 와 `range()` 로 정해진 횟수만큼 반복
- `while` 로 조건이 참인 동안 반복
- `break` (멈춤) 와 `continue` (건너뛰기)
- 중첩 반복으로 구구단 만들기

## 7.1 for 와 range() — 정해진 횟수만큼

`for` 반복문은 '정해진 횟수'나 '목록의 항목들'을 하나씩 훑을 때 써요. 먼저 `range()` 와 함께 쓰는 형태를 볼게요. `range(5)` 는 0, 1, 2, 3, 4 (0부터 5개)를 만들어 줘요.

파이썬 코드

```
for i in range(5):
    print(i)
```

실행 결과

```
0
1
2
3
4
```

`i` 라는 변수에 0, 1, 2, 3, 4가 차례로 담기면서 각각 출력됐어요. `for` 도 조건문처럼 콜론 `:` 과 들여쓰기를 써요. 들여쓰기된 부분이 '반복할 내용'이에요.

`range()` 에 값을 두 개 주면 시작과 끝을 정할 수 있어요. 이때도 끝 숫자는 포함하지 않아요. `range(1, 6)` 은 1, 2, 3, 4, 5예요.

파이썬 코드

```
for i in range(1, 6):
    print(i, "번째 인사: 안녕!")
```

#### 실행 결과

```
1 번째 인사: 안녕!  
2 번째 인사: 안녕!  
3 번째 인사: 안녕!  
4 번째 인사: 안녕!  
5 번째 인사: 안녕!
```



#### range() 정리

- `range(5)` → 0,1,2,3,4 (0부터 5개)
  - `range(1, 6)` → 1,2,3,4,5 (1부터 6 전까지)
  - `range(0, 10, 2)` → 0,2,4,6,8 (2칸씩 건너뛰기)
- 끝 숫자는 항상 포함하지 않는다는 걸 기억하세요.

### ▶ 리스트를 하나씩 꺼내기

`for` 는 리스트 같은 묶음의 항목을 하나씩 꺼낼 때도 써요. (리스트는 다음 장에서 자세히!) 이게 `for` 의 진짜 강점이에요.

#### 파이썬 코드

```
fruits = ["사과", "바나나", "포도"]  
for fruit in fruits:  
    print("나는", fruit, "를 좋아해")
```

#### 실행 결과

```
나는 사과 를 좋아해  
나는 바나나 를 좋아해  
나는 포도 를 좋아해
```

`fruits` 안의 값이 하나씩 `fruit` 에 담기면서 반복돼요. 번호를 몰라도 묶음의 모든 항목을 자동으로 훑어 주니 편하죠.

## 7.2 while — 조건이 참인 동안

`while`은 '~하는 동안'이라는 뜻이에요. 반복 횟수가 정해지지 않고 어떤 조건이 참인 동안 계속 반복할 때 써요. 조건이 거짓이 되면 멈춰요.

#### 파이썬 코드

```
count = 3  
while count > 0:  
    print(count)  
    count -= 1  
print("발사!")
```

실행 결과

```
3
2
1
발사!
```

`count` 가 3에서 시작해, `count > 0` 인 동안 출력하고 1씩 줄여요. 3 → 2 → 1 을 거쳐 `count` 가 0이 되면 조건이 거짓이라 반복을 빠져나와 '발사!'를 출력해요.



#### 무한 반복 주의!

`while` 에서 조건을 거짓으로 만드는 코드( `count -= 1` 같은)를 빠뜨리면, 조건이 영원히 참이라 프로그램이 멈추지 않아요(무한 루프). 이러면 `Ctrl + C` 로 강제 종료해야 해요. `while` 안에는 반드시 '언젠가 조건을 거짓으로 만들 코드'가 있어야 해요.

## 7.3 break 와 continue — 반복 제어하기

반복 도중에 흐름을 바꾸는 두 가지 명령이 있어요.

- `break` — 반복을 즉시 완전히 멈춰요. 반복문을 탈출해요.
- `continue` — 이번 회차만 건너뛰고 다음 회차로 넘어가요.

### ▶ break: 조건을 만나면 탈출

파이썬 코드

```
for i in range(1, 11):
    if i == 5:
        break
    print(i)
```

실행 결과

```
1
2
3
4
```

원래 1부터 10까지 둘 예정이었지만, `i` 가 5가 되는 순간 `break` 로 반복을 완전히 빠져나왔어요. 그래서 5는 출력조차 안 되고 1~4까지만 나왔죠.



## ▶ continue: 특정 값만 건너뛰기

파이썬 코드

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

실행 결과

```
1  
2  
4  
5
```

`i` 가 3일 때 `continue` 를 만나 그 회차의 `print` 를 건너뛰었어요. 그래서 3만 빠지고 나머지는 다 출력됐죠. `break` 는 '멈춤', `continue` 는 '이번 것만 패스'라고 기억하세요.

## 7.4 중첩 반복 — 구구단 만들기

반복문 안에 반복문을 넣으면 **중첩 반복**이 돼요. 바깥 반복이 한 번 돌 때마다 안쪽 반복이 통째로 다 돌아요. 구구단이 딱 이 구조예요: 바깥은 '단', 안쪽은 '곱하는 수'.

파이썬 코드

```
for dan in range(2, 4):  
    print(f"--- {dan}단 ---")  
    for i in range(1, 10):  
        print(f"{dan} x {i} = {dan * i}")
```

#### 실행 결과

```
---- 2단 ----
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
---- 3단 ----
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
```

바깥 `for (dan)`가 2, 3으로 돌고, 각 단마다 안쪽 `for (i)`가 1~9로 돌아요. (지면 관계로 2·3단만 예로 들었어요. `range(2, 10)`으로 바꾸면 9단까지 나와요.) f-string으로 계산 결과까지 깔끔하게 넣었죠.

## 7.5 실전: 1부터 10까지 합 구하기

반복문의 단골 활용이 '누적'이에요. 합을 담을 상자를 0으로 시작해 두고, 반복하며 `+=`로 더해 나가면 돼요.

#### 파이썬 코드

```
total = 0
for i in range(1, 11):
    total += i
print("1부터 10까지 합:", total)
```

#### 실행 결과

```
1부터 10까지 합: 55
```

`total`이 0에서 시작해 1, 2, 3... 10을 차례로 더해 최종 55가 됐어요. 이 '초기값 → 반복하며 누적' 패턴은 앞으로 정말 자주 써요. 3장의 `+=`가 여기서 빛을 발하죠.

## 연습문제

1. `for` 와 `range()` 로 "안녕!" 을 5번 출력해 보세요.
2. 1부터 100까지의 짝수만 더한 합을 출력해 보세요. (힌트: `range(2, 101, 2)` )
3. 1부터 20까지 반복하되, 7이 나오면 `break` 로 멈추는 코드를 만들어 어디까지 출력되는지 확인해 보세요.

---

### 해답 · 힌트

② `total = 0` 후 `for i in range(2, 101, 2): total += i` → 합은 2550 . ③ 1,2,3,4,5,6 까지 출력되고 7에서 멈춤(7은 출력 안 됨).

# 자료구조

값 하나가 아니라 여러 값을 묶어서 다뤄야 할 때가 많아요. 학생 30명의 점수, 장바구니 물건들처럼요. 이렇게 여러 값을 담는 '그릇'을 자료구조라고 해요. 파이썬엔 성격이 다른 네 가지 그릇이 있어요. 각각 언제 쓰면 좋은지까지 익혀 두면 앞으로 코드를 훨씬 잘 짤 수 있어요.

## 이 장에서 배울 것

- 리스트: 순서 있는 값 묶음(추가·삭제·정렬·슬라이싱)
- 튜플: 바뀌지 않는 묶음
- 딕셔너리: 이름표(키)로 값 찾기
- 셋: 중복 없는 집합, 어떤 걸 언제 쓸까

## 8.1 리스트 — 가장 많이 쓰는 값 묶음

**리스트(list)**는 여러 값을 **순서대로** 담는 그릇이에요. 대괄호 `[ ]` 로 만들고, 값을 쉼표로 구분해요. 4장에서 배운 문자열 인덱싱처럼, 리스트도 **0번부터** 번호로 값을 꺼내요.

파이썬 코드

```
fruits = ["사과", "바나나", "포도"]
print(fruits)
print(fruits[0])
print(fruits[-1])
print(len(fruits))
```

실행 결과

```
['사과', '바나나', '포도']
사과
포도
3
```

`fruits[0]` 은 첫 값 '사과', `fruits[-1]` 은 마지막 '포도'예요. `len()` 은 리스트에 값이 몇 개 들었는지(길이) 알려 줘요. 여기서 3개죠.

### ▶ 값 추가·삭제하기

리스트는 값을 자유롭게 넣고 뺄 수 있어요. 이게 리스트의 큰 장점이예요.

파이썬 코드

```
fruits = ["사과", "바나나"]
fruits.append("포도")      # 맨 뒤에 추가
print(fruits)
fruits.insert(0, "딸기")  # 0번 자리에 끼워넣기
print(fruits)
fruits.remove("바나나")   # 값으로 삭제
print(fruits)
```

실행 결과

```
['사과', '바나나', '포도']
['딸기', '사과', '바나나', '포도']
['딸기', '사과', '포도']
```

- `.append(값)` — 맨 뒤에 값을 추가.
- `.insert(위치, 값)` — 원하는 위치에 끼워 넣기.
- `.remove(값)` — 그 값을 찾아 삭제.

## ▶ 정렬하기

파이썬 코드

```
nums = [3, 1, 4, 1, 5, 9, 2]
nums.sort()          # 오름차순 정렬
print(nums)
nums.sort(reverse=True) # 내림차순 정렬
print(nums)
```

실행 결과

```
[1, 1, 2, 3, 4, 5, 9]
[9, 5, 4, 3, 2, 1, 1]
```

`.sort()` 는 리스트를 작은 값부터 정렬해요. `reverse=True` 옵션을 주면 큰 값부터 정렬하죠.

## ▶ 슬라이싱 — 리스트도 잘라요

문자열처럼 리스트도 `[시작:끝]` 슬라이싱이 돼요. 끝 번호는 포함하지 않는 규칙도 같아요.

파이썬 코드

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])
print(nums[:2])
print(nums[3:])
```

실행 결과

```
[20, 30, 40]
[10, 20]
[40, 50]
```



리스트는 7장 `for` 반복문과 찰떡궁합이에요. `for item in fruits:` 처럼 쓰면 리스트의 모든 값을 하나씩 꺼내 처리할 수 있어요. 여러 값을 다룰 땐 `리스트 + 반복문` 조합을 가장 많이 씁니다.

## 8.2 튜플 — 바뀌지 않는 묶음

튜플(tuple)은 리스트와 비슷하지만 `한 번 만들면 값을 바꿀 수 없어요`. 소괄호 `( )` 로 만들어요. 좌표나 색상값처럼 '절대 바뀌면 안 되는 값'을 담을 때 써요.

파이썬 코드

```
point = (37, 127)
print(point[0])
print(point[1])
```

실행 결과

```
37
127
```

값을 꺼내는 방법은 리스트와 같아요(번호로). 다만 `point[0] = 40` 처럼 값을 바꾸려 하면 오류가 나요. '이건 바뀌면 안 돼'라고 못 박아 두는 안전장치인 셈이에요.



### 리스트 vs 튜플

- **리스트** `[ ]` — 값을 바꿀 수 있음. 대부분의 경우 이걸 써요.
- **튜플** `( )` — 값을 바꿀 수 없음. 고정된 값 묶음에.

## 8.3 딕셔너리 — 이름표로 값 찾기

딕셔너리(dictionary, 사전)는 번호 대신 **이름표(키)**로 값을 찾는 그릇이에요. 사전에서 단어(키)를 찾으면 뜻(값)이 나오듯이요. `{키: 값}` 형태로 중괄호 `{ }` 안에 짝지어 넣어요.

파이썬 코드

```
scores = {"국어": 90, "영어": 85, "수학": 95}
print(scores["수학"])      # 키로 값 찾기
scores["과학"] = 88       # 새 항목 추가
print(scores)
```

실행 결과

```
95
{'국어': 90, '영어': 85, '수학': 95, '과학': 88}
```

`scores["수학"]` 처럼 키를 넣으면 그 값이 나와요. 없는 키에 값을 넣으면( `scores["과학"] = 88` ) 새 항목이 추가돼요. 번호가 아니라 '의미 있는 이름'으로 값을 관리하니 데이터가 많아도 헷갈리지 않아요.

#### ▶ 딕셔너리 전체 훑기: `items()`

파이썬 코드

```
scores = {"국어": 90, "영어": 85}
for subject, score in scores.items():
    print(f"{subject}: {score}점")
```

실행 결과

```
국어: 90점
영어: 85점
```

`.items()` 를 `for` 와 함께 쓰면 키와 값을 한 번에 꺼내며 반복할 수 있어요. `subject` 엔 키가, `score` 엔 값이 담겨요.

## 8.4 셋 — 중복 없는 집합

셋(set, 집합)은 중복을 허용하지 않는 그릇이에요. 같은 값을 여러 번 넣어도 하나만 남아요. 중복을 없앨 때 아주 편해요. 중괄호 `{ }` 로 만들거나 `set()` 으로 만들어요.

파이썬 코드

```
numbers = [1, 2, 2, 3, 3, 3, 4]
unique = set(numbers)  # 리스트의 중복 제거
print(unique)
```

실행 결과

```
{1, 2, 3, 4}
```

중복된 2와 3이 하나씩만 남았어요. 수학의 집합처럼 교집합·합집합도 구할 수 있어요.

파이썬 코드

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a & b)    # 교집합
print(a | b)    # 합집합
```

실행 결과

```
{3}
{1, 2, 3, 4, 5}
```

`&` 는 양쪽에 다 있는 값(교집합), `|` 는 둘을 합친 값(합집합)이에요. '공통 관심사 찾기'나 '전체 목록 합치기' 같은 데 써먹을 수 있죠.

## 8.5 무엇을 언제 쓸까?

| 자료구조 | 기호                 | 특징               | 이럴 때 써요                 |
|------|--------------------|------------------|-------------------------|
| 리스트  | <code>[ ]</code>   | 순서 O, 변경 O, 중복 O | 가장 흔한 값 묶음(장바구니, 점수 목록) |
| 튜플   | <code>( )</code>   | 순서 O, 변경 X       | 바뀌면 안 되는 값(좌표, 설정값)     |
| 딕셔너리 | <code>{키:값}</code> | 이름표로 관리          | 이름과 값이 짝지어진 데이터(회원 정보)  |
| 셋    | <code>{ }</code>   | 중복 X, 순서 X       | 중복 제거, 집합 연산            |



헛갈리면 **일단 리스트**로 시작하세요. 실제로 리스트를 가장 많이 써요. '중복을 없애야겠다' 싶으면 셋, '이름으로 찾고 싶다' 싶으면 딕셔너리를 떠올리면 돼요.

### 연습문제

1. 좋아하는 영화 3개를 담은 리스트를 만들고, `append()` 로 하나 더 추가한 뒤 전체와 개수( `len` )를 출력해 보세요.
2. 딕셔너리로 자신의 정보( "이름", "나이", "도시" )를 만들고, 각 값을 키로 꺼내 출력해 보세요.
3. 리스트 `[1, 1, 2, 3, 3, 3]` 의 중복을 `set()` 으로 제거하면 무엇이 남을까요?

### 해답 · 힌트

② 예: `me = {"이름": "지우", "나이": 8, "도시": "서울"}` 후 `print(me["이름"])` 등. ③ `{1, 2, 3}` — 중복이 사라진 세 값만 남아요.



# 함수

같은 코드를 여기저기 반복해서 쓰다 보면 길어지고, 고칠 때도 여러 군데를 다 바꿔야 해요. 이럴 때 코드에 '이름'을 붙여 하나로 묶어 두는 게 함수예요. 한 번 만들어 두면 이름만 부르면 되죠. 함수는 프로그램을 정리하고 재사용하게 해 주는 아주 중요한 개념이에요.

## 이 장에서 배울 것

- 함수가 왜 필요한지, `def` 로 만들기
- `return` 으로 결과 돌려받기
- 매개변수·인자, 기본값, 키워드 인자
- 지역 변수와 전역 변수(스코프) 알아보기

## 9.1 함수란? — 코드에 이름 붙이기

**함수(function)**는 특정 일을 하는 코드 묶음에 이름을 붙인 것이예요. 요리에 비유하면, '라면 끓이기'라는 이름 아래 여러 단계를 묶어 두는 거죠. 이제 '라면 끓이기'라고 부르기만 하면 그 안의 모든 단계가 실행돼요. 함수는 `def 이름():` 으로 만들고, 부를 땐 `이름()` 으로 불러요(호출).

파이썬 코드

```
def say_hello():
    print("안녕하세요!")
    print("반갑습니다!")

say_hello()    # 함수 부르기(호출)
say_hello()    # 또 부르기
```

실행 결과

```
안녕하세요!
반갑습니다!
안녕하세요!
반갑습니다!
```

`def say_hello():` 로 함수를 정의만 해 두면 아직 실행되지 않아요. `say_hello()` 라고 불러야 비로소 안쪽 코드가 실행돼요. 한 번 만들어 두고 두 번 불렀더니 인사가 두 번 나왔죠. 이게 '재사용'이에요.



### 정의와 호출은 달라요

- **정의**(`def ...`): 함수를 만들어 두는 것. 아직 실행 안 됨.
  - **호출**(`이름()`): 만들어 둔 함수를 실제로 실행하는 것.
- 레시피를 적어 두는 것(정의)과 실제로 요리하는 것(호출)의 차이예요.

## 9.2 매개변수와 인자 — 함수에 값 건네기

함수에 값을 넘겨주면 그 값으로 일을 하게 만들 수 있어요. 괄호 안에 받을 값의 이름을 적는데, 이것 **매개변수(parameter)**라고 해요. 함수를 부를 때 실제로 넣는 값은 **인자(argument)**라고 하고요.

파이썬 코드

```
def add(a, b):
    return a + b

result = add(3, 5)
print(result)
print(add(10, 20))
```

실행 결과

```
8
30
```

`add(a, b)` 는 두 값을 받는 함수예요. `add(3, 5)` 라고 부르면 `a` 엔 3, `b` 엔 5가 담기고, `a + b` 인 8을 돌려줘요 (`return`). 돌려준 값은 `result` 에 담아 쓰거나 바로 출력할 수 있어요.

### ▶ return — 결과를 돌려주기

`return` 은 함수의 결과를 **바깥으로 돌려주는** 명령이에요. `print` 가 '화면에 보여 주기'라면, `return` 은 '값을 넘겨주기'예요. 돌려받은 값은 변수에 담거나 다른 계산에 쓸 수 있죠. `return` 을 만나면 함수는 그 즉시 끝나요.

## 9.3 기본값과 키워드 인자

매개변수에 **기본값**을 정해 두면, 인자를 안 줬을 때 그 값이 자동으로 쓰여요. **매개변수=기본값** 형태로 적어요.

파이썬 코드

```
def greet(name, greeting="안녕"):
    print(f"{greeting}, {name}님!")

greet("지우")           # greeting 생략 → 기본값 "안녕"
greet("민수", "반가워") # greeting 직접 지정
```

#### 실행 결과

안녕, 지우님!  
반가워, 민수님!

`greet("지우")` 는 `greeting` 을 안 줘서 기본값 '안녕'이 쓰였고, `greet("민수", "반가워")` 는 직접 준 '반가워'가 쓰였어요.

### ▶ 키워드 인자 — 이름을 붙여 건네기

인자를 넘길 때 `매개변수=값` 처럼 이름을 붙이면 순서를 신경 쓰지 않아도 돼요. 이것 **키워드 인자**라고 해요. 5장에서 본 `sep=`, `end=` 가 바로 이거였어요!

#### 파이썬 코드

```
def introduce(name, age):  
    print(f"{name}는 {age}살입니다.")  
  
introduce(age=8, name="지우")  # 순서를 바꿔도 이름으로 정확히 전달
```

#### 실행 결과

지우는 8살입니다.

이름을 붙였기 때문에 `age` 를 먼저 써도 각 값이 제자리를 찾아가요. 인자가 많을 때 헷갈리지 않게 해 주는 편리한 방법이에요.

## 9.4 여러 개의 인자 받기 — \*args 맛보기

몇 개가 올지 모르는 값을 한꺼번에 받고 싶을 땐 매개변수 앞에 별표 `*` 를 붙여요. 그러면 넘어온 값들이 모두 하나의 묶음(튜플)으로 들어와요. 관례로 이름을 `*args` 라고 많이 써요.

#### 파이썬 코드

```
def total(*numbers):  
    return sum(numbers)  
  
print(total(1, 2, 3))  
print(total(10, 20, 30, 40))
```

#### 실행 결과

6  
100

`total(1, 2, 3)` 은 세 값을, `total(10, 20, 30, 40)` 은 네 값을 받아 각각 합을 돌려줘요. `sum()` 은 묶음의 값을 모두 더해 주는 파이썬 내장 함수예요. 인자 개수에 얽매이지 않는 유연한 함수를 만들 수 있죠.



**\*\*kwargs** 라는 형태도 있어요. 이건 이름 붙은 인자들을 딕셔너리로 받아요. 지금은 '별 하나( \* )는 값 묶음, 별 둘( \*\* )은 이름-값 묶음'이라는 정도만 알아 두세요. 나중에 자연스럽게 익히게 돼요.

## 9.5 지역 변수와 전역 변수 — 스코프

변수에는 '통하는 범위'가 있어요. 이것을 **스코프(scope)**라고 해요. 함수 안에서 만든 변수는 그 함수 안에서만 통하고(지역 변수), 함수 밖에서 만든 변수는 어디서나 통해요(전역 변수).

파이썬 코드

```
x = 10 # 전역 변수(함수 밖)

def show():
    x = 99 # 지역 변수(이 함수 안에서만)
    print("함수 안:", x)

show()
print("함수 밖:", x)
```

실행 결과

```
함수 안: 99
함수 밖: 10
```

함수 안에서 만든 `x = 99` 는 그 함수 안에서만 존재하는 다른 상자예요. 그래서 함수 밖의 `x` 는 여전히 10으로 남아 있어요. 이름이 같아도 서로 다른 상자라는 게 핵심이에요. 이 덕분에 함수끼리 변수 이름이 겹쳐도 서로 방해하지 않아요.



### 함수 안 변수는 밖에서 못 써요

함수 안에서만 만든 지역 변수를 함수 밖에서 쓰려고 하면 '그런 변수 없다'는 오류가 나요. 함수의 결과를 밖에서 쓰고 싶으면 `return` 으로 값을 돌려받아 쓰세요. 이게 함수를 안전하게 쓰는 올바른 방법이에요.

### 연습문제

- 이름을 받아 "00님 환영합니다!" 를 출력하는 함수 `welcome(name)` 을 만들어 두 사람에게 인사해 보세요.
- 두 수를 받아 큰 값을 `return` 하는 함수를 만들어 보세요. (힌트: 조건문으로 비교, 또는 `max()` )
- `*args` 를 이용해 넘어온 값들의 개수를 돌려주는 함수를 만들어 보세요. (힌트: `len()` )

#### 해답 · 힌트

② 예: `def bigger(a, b): return a if a > b else b` 또는 `return max(a, b)` . ③ `def count(*args): return len(args) - count(1,2,3)` 은 3.

# 모듈과 표준 라이브러리 맛보기

파이썬이 인기 있는 큰 이유 중 하나는 '이미 만들어진 도구'가 어마어마하게 많다는 거예요. 수학 계산, 무작위 뽑기, 날짜 다루기... 우리가 직접 안 만들어도 불러다 쓰면 돼요. 이렇게 기능을 모아 둔 파일을 모듈이라 하고, 파이썬에 기본 포함된 모듈 묶음을 표준 라이브러리라고 해요.

## 이 장에서 배울 것

- `import` 로 모듈 불러오기
- `math` : 수학 계산 도구
- `random` : 무작위 뽑기
- `datetime` : 날짜와 시간 다루기

## 10.1 모듈이란? — 남이 만든 도구 상자

**모듈(module)**은 유용한 함수와 기능을 미리 모아 둔 '도구 상자'예요. 필요한 도구 상자를 `import 모듈이름` 으로 불러온 뒤, `모듈이름.기능()` 형태로 그 안의 도구를 꺼내 써요. 파이썬을 설치하면 유용한 모듈들이 이미 함께 깔려 있어요(표준 라이브러리).

## 10.2 math — 수학 계산 도구

`math` 모듈에는 제곱근, 올림·내림, 원주율 같은 수학 도구가 들어 있어요.

파이썬 코드

```
import math

print(math.sqrt(16))    # 제곱근
print(math.pi)         # 원주율
print(math.ceil(3.2))   # 올림
print(math.floor(3.8))  # 내림
```

실행 결과

```
4.0
3.141592653589793
4
3
```

- `math.sqrt(16)` — 16의 제곱근 `4.0`.
- `math.pi` — 원주율  $\pi$  값(약 3.14159...). 괄호 없이 값 그 자체예요.

- `math.ceil(3.2)` — 무조건 올림 → 4.
- `math.floor(3.8)` — 무조건 내림 → 3.



2장에서 `int(3.9)` 가 3이 된 걸 기억하죠? 그건 소수점 아래를 '버리는' 거예요. 반면 `math.ceil` 은 무조건 위로, `math.floor` 는 무조건 아래로 정수를 만들어요. 필요에 맞게 골라 쓰세요.

## 10.3 random — 무작위 뽑기

`random` 모듈은 주사위 굴리기, 제비뽑기처럼 무작위 결과를 만들 때 써요. 게임이나 추첨 프로그램에 딱이에요.

파이썬 코드

```
import random

print(random.randint(1, 6))           # 1~6 중 무작위 정수
print(random.choice(["가위", "바위", "보"])) # 목록에서 무작위로 하나
```

실행 결과

```
6
가위
```

- `random.randint(1, 6)` — 1부터 6까지 중 무작위 정수(주사위!). 여기서 6이 나왔어요.
- `random.choice(목록)` — 리스트에서 무작위로 하나 뽑기. 여기서 '가위'가 뽑혔죠.



**무작위라 결과가 매번 달라요**

`random` 은 실행할 때마다 결과가 바뀌는 게 정상이에요! 그래서 이 책의 결과와 여러분의 실행 결과가 다를 수 있어요. (참고로 이 책은 결과를 고정하려고 내부적으로 `random.seed()` 라는 걸 썼어요. 여러분은 그냥 실행해 보며 매번 달라지는 걸 즐기세요.)

## 10.4 datetime — 날짜와 시간

`datetime` 모듈로 날짜와 시간을 다룰 수 있어요. 특정 날짜를 만들어 연·월·일을 따로 꺼낼 수도 있죠.

파이썬 코드

```
import datetime

d = datetime.date(2026, 8, 11) # 특정 날짜 만들기
print(d)
print(d.year, d.month, d.day)
```

실행 결과

```
2026-08-11
2026 8 11
```

`datetime.date(2026, 8, 11)` 로 날짜를 만들고, `.year`, `.month`, `.day` 로 연·월·일을 각각 꺼냈어요. 오늘 날짜가 궁금하면 `datetime.date.today()` 를 쓰면 돼요(실행하는 날에 따라 달라져요).

## 10.5 필요한 것만 콕 집어 불러오기: `from ~ import`

모듈 전체가 아니라 특정 기능 하나만 쓰고 싶을 땐 `from 모듈 import 기능` 형태를 써요. 그러면 `모듈이름.` 을 앞에 붙이지 않고 바로 쓸 수 있어요.

파이썬 코드

```
from math import sqrt

print(sqrt(81))  # math.sqrt 가 아니라 sqrt 로 바로 사용
```

실행 결과

```
9.0
```

`from math import sqrt` 덕분에 `math.` 없이 `sqrt(81)` 만 써도 됐어요. 자주 쓰는 기능 몇 개만 가져올 때 편해요.



### import 방법 정리

- `import math` → `math.sqrt(9)` 처럼 모듈 이름을 붙여 사용.
- `from math import sqrt` → `sqrt(9)` 처럼 바로 사용.

둘 다 자주 쓰이니 눈에 익혀 두세요.



파이썬 표준 라이브러리에는 이 밖에도 수백 개의 모듈이 있어요. 인터넷 통신, 파일 압축, 데이터 저장 등 웬만한 건 다 있죠. 그리고 표준에 없는 기능도 `pip` 라는 도구로 설치해 쓸 수 있는데, 이건 2권에서 다뤄요. 지금은 '필요한 도구는 대개 이미 누군가 만들어 놨다'는 걸 아는 것만으로 충분해요.

## 연습문제

1. `math` 를 이용해 100의 제곱근을 출력해 보세요. 결과는 무엇일까요?
2. `random.randint(1, 100)` 을 세 번 출력해 보세요. 값이 매번 같나요, 다른가요?
3. `from random import choice` 로 좋아하는 과일 리스트에서 오늘의 과일을 하나 뽑아 출력해 보세요.

---

### 해답 · 힌트

- ① `math.sqrt(100)` → `10.0` (제곱근은 실수로 나와요). ② 무작위라 대개 매번 다른 값이 나와요. ③ 예: `print(choice(["사과", "귤", "포도"]))`.



# 파일 입출력

프로그램을 끄면 변수에 담긴 값들은 모두 사라져요. 하지만 메모나 기록은 남겨 두고 싶을 때가 있죠. 그럴 때 값을 '파일'에 저장해요. 이 장에서는 텍스트 파일을 안전하게 만들고, 쓰고, 읽는 법을 배워요. 특히 파일을 다룰 때 꼭 써야 하는 `with` 문을 익히면 실수 없이 파일을 다룰 수 있어요.

## 이 장에서 배울 것

- `open()` 으로 파일 열기(읽기·쓰기 모드)
- `with` 문으로 안전하게 다루기
- 파일에 쓰기(write)와 읽기(read)
- 한 줄씩 읽기, 이어 쓰기(append)

## 11.1 파일 쓰기 — with 문으로 안전하게

파일을 다루는 기본은 `open(파일이름, 모드)` 예요. 모드는 무엇을 할지 정하는 글자예요: `"w"` (쓰기), `"r"` (읽기), `"a"` (이어 쓰기). 그런데 파일은 다 쓰고 나면 꼭 닫아 줘야 해요. 안 닫으면 내용이 제대로 저장되지 않을 수 있거든요. 이 '닫기'를 자동으로 해 주는 게 `with` 문이에요.

파이썬 코드

```
with open("memo.txt", "w", encoding="utf-8") as f:
    f.write("파이썬 공부 시작!\n")
    f.write("오늘도 화이팅\n")
print("파일 저장 완료")
```

실행 결과

파일 저장 완료

`with open(...) as f:` 는 파일을 열어 `f` 라는 이름으로 다루겠다는 뜻이에요. 안쪽에서 `f.write()` 로 내용을 써요. `with` 블록이 끝나면 파일이 자동으로 닫혀요. 그래서 실수로 안 닫는 일이 없어요.



### 왜 with 를 쓰나요?

`with` 없이 파일을 열면 다 쓴 뒤 `f.close()` 로 직접 닫아야 하는데, 이것을 깜빡하기 쉬워요. `with` 를 쓰면 블록을 벗어날 때 알아서 닫아 주니 안전하고 편해요. 파일은 **항상 with 로** 다루는 습관을 들이세요.



### "w" 모드는 기존 내용을 지워요!

쓰기 모드 (`"w"`)로 열면 그 파일에 원래 있던 내용은 모두 사라지고 새로 씁니다. 기존 내용을 유지하며 뒤에 덧붙이고 싶으면 이어 쓰기 모드 (`"a"`)를 쓰세요(뒤에서 배워요).



`encoding="utf-8"` 은 한글이 깨지지 않게 해 주는 설정이에요. 한글이 든 파일을 다룰 땐 이 옵션을 붙이는 걸 권해요. 그리고 `\n` 은 4장에서 배운 줄바꿈 신호였죠? 파일에 줄을 나눠 쓸 때 써요.

## 11.2 파일 읽기 — read

저장한 파일을 다시 열어 내용을 읽어 볼게요. 이번엔 읽기 모드 `"r"` 로 열고 `f.read()` 로 전체 내용을 한 번에 읽어요.

파이썬 코드

```
with open("memo.txt", "r", encoding="utf-8") as f:
    content = f.read()
print(content)
```

실행 결과

```
파이썬 공부 시작!
오늘도 화이팅
```

`f.read()` 가 파일 전체를 하나의 문자열로 읽어 `content` 에 담았어요. 앞서 `\n` 으로 줄을 나눠 썼기 때문에 출력도 두 줄로 나왔죠.

## 11.3 한 줄씩 읽기

파일이 길면 전체를 한꺼번에 읽기보다 한 줄씩 처리하는 게 좋을 때가 있어요. `for` 반복문에 파일을 넣으면 줄 단위로 하나씩 꺼내 줘요.

파이썬 코드

```
with open("memo.txt", "r", encoding="utf-8") as f:
    for line in f:
        print("읽은 줄:", line.strip())
```

실행 결과

```
읽은 줄: 파이썬 공부 시작!
읽은 줄: 오늘도 화이팅
```

`for line in f:` 는 파일을 한 줄씩 `line` 에 담아 반복해요. 각 줄 끝에는 줄바꿈(`\n`)이 붙어 있어서, 4장에서 배운 `.strip()` 으로 양 끝 공백·줄바꿈을 없앴어요. 그래서 깔끔하게 출력됐죠.

## 11.4 이어 쓰기 — append 모드

기존 내용을 지우지 않고 뒤에 덧붙이려면 이어 쓰기 모드 `"a"` (append)를 써요. 로그(기록)를 쌓을 때 유용해요.

#### 파이썬 코드

```
with open("memo.txt", "a", encoding="utf-8") as f:
    f.write("한 줄 더 추가\n")

with open("memo.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

#### 실행 결과

```
파이썬 공부 시작!
오늘도 화이팅
한 줄 더 추가
```

"a" 모드로 열어 새 줄을 덧붙이니, 기존 두 줄은 그대로 있고 맨 아래에 '한 줄 더 추가'가 붙었어요. "w" 였다면 앞 내용이 다 지워졌을 거예요.

| 모드  | 이름            | 하는 일               |
|-----|---------------|--------------------|
| "r" | 읽기(read)      | 파일 내용을 읽어요(기본값)    |
| "w" | 쓰기(write)     | 새로 써요. 기존 내용은 지워짐! |
| "a" | 이어 쓰기(append) | 기존 내용 뒤에 덧붙여요      |



파일 이름만 주고 경로를 안 쓰면, 프로그램을 실행한 현재 폴더에 파일이 만들어져요. 특정 위치에 저장하고 싶으면 "C:/폴더/memo.txt" 처럼 전체 경로를 주면 돼요. 읽으려는 파일이 없으면 오류가 나는데, 이런 오류를 다루는 법은 바로 다음 12장에서 배워요.

#### 연습문제

1. `diary.txt` 파일을 만들어 오늘 있었던 일 두 줄을 써 보세요. (힌트: "w", \n)
2. 방금 만든 `diary.txt` 를 읽어 화면에 전체 출력해 보세요. (힌트: "r", read())
3. "a" 모드로 `diary.txt` 에 한 줄을 더 추가한 뒤 다시 읽어, 세 줄이 되었는지 확인해 보세요.

#### 해답 · 힌트

모두 `with open(...) as f:` 형태로 다루세요. ①은 `f.write("...\n")`, ②는 `print(f.read())`, ③은 "a" 모드로 열어 `f.write` 하면 앞 두 줄 뒤에 붙어요.

# 예외 처리

프로그램을 짜다 보면 오류(에러)를 정말 많이 만나요. 그건 실력이 부족해서가 아니라 지극히 자연스러운 일이에요! 중요한 건 오류가 났을 때 프로그램이 그냥 죽어 버리지 않고 '괜찮아, 이렇게 대처할게' 하도록 만드는 거예요. 이 장에서 오류를 다루는 법과, 에러 메시지를 겁내지 않고 읽는 법을 배워요.

## 이 장에서 배울 것

- 오류가 나면 프로그램이 어떻게 되는지
- `try · except` 로 오류에 대처하기
- `finally` 로 마무리 작업 보장하기
- 자주 만나는 에러 종류와 메시지 읽는 법

## 12.1 오류는 나쁜 게 아니에요

파이썬은 문제가 생기면 **에러 메시지**를 보여 주고 프로그램을 멈춰요. 예를 들어 글자 `"abc"` 를 숫자로 바꾸려 하면 바꿀 수가 없어서 오류가 나죠. 이렇게 실행 중에 발생하는 오류를 **예외(exception)**라고 불러요. 에러 메시지는 **혼내는 게 아니라 '여기에 이런 문제가 있어요'라고 알려 주는 친절한 안내**예요.

에러 메시지의 **마지막 줄**이 가장 중요해요. 거기에 '무슨 종류의 오류'인지 적혀 있거든요. 예를 들어 `ValueError: invalid literal for int()...` 라면 앞부분 `ValueError` 가 오류 종류예요.

| 에러 이름                          | 이런 상황에서 나요                               |
|--------------------------------|------------------------------------------|
| <code>ValueError</code>        | 값이 잘못됨(예: <code>int("abc")</code> )      |
| <code>ZeroDivisionError</code> | 0으로 나눔(예: <code>10 / 0</code> )          |
| <code>TypeError</code>         | 자료형이 안 맞음(예: 문자열 + 숫자)                   |
| <code>IndexError</code>        | 리스트에 없는 번호를 꺼냄(예: <code>lst[99]</code> ) |
| <code>NameError</code>         | 없는 변수를 씀(오타 등)                           |
| <code>FileNotFoundError</code> | 없는 파일을 열려고 함                             |



에러 메시지가 영어라 겁날 수 있지만, 사실 몇 종류가 반복해서 나와요. 위 표의 이름들만 눈에 익혀 두면 대부분의 오류를 금방 알아볼 수 있어요. 메시지를 그대로 검색창에 복사해 넣으면 해결법도 쉽게 찾을 수 있고요.

## 12.2 try · except — 오류에 대처하기

오류가 날 만한 코드를 `try:` 블록에 넣고, 오류가 났을 때 할 일을 `except:` 블록에 적어요. 그러면 프로그램이 죽는 대신 `except` 쪽으로 넘어가 우아하게 대처해요.

파이썬 코드

```
try:
    num = int("abc")    # 여기서 오류 발생!
    print(num)
except ValueError:
    print("숫자로 바꿀 수 없어요!")
```

실행 결과

숫자로 바꿀 수 없어요!

`int("abc")` 에서 `ValueError` 가 나자, 프로그램이 멈추는 대신 `except` 로 넘어가 안내 메시지를 출력했어요. 이렇게 하면 사용자가 이상한 값을 입력해도 프로그램이 갑자기 죽지 않아요.

파이썬 코드

```
try:
    result = 10 / 0    # 0으로 나눔 → 오류
except ZeroDivisionError:
    print("0으로 나눌 수 없어요!")
```

실행 결과

0으로 나눌 수 없어요!

`except` 뒤에 오류 종류를 적으면 그 오류가 났을 때만 대처해요. 종류를 안 적으면 모든 오류를 잡지만, 어떤 오류인지 구분하려면 종류를 적는 게 좋아요.

## 12.3 finally — 어떤 경우든 꼭 실행

`finally:` 블록은 오류가 나든 안 나든 무조건 마지막에 실행돼요. 파일을 닫거나 정리 작업처럼 '무슨 일이 있어도 꼭 해야 하는 일'에 써요.

파이썬 코드

```
try:
    print("시도합니다")
    x = 1 / 0
except ZeroDivisionError:
    print("오류 발생!")
finally:
    print("어쨌든 이걸 항상 실행돼요.")
```

#### 실행 결과

```
시도합니다
오류 발생!
어쨌든 이건 항상 실행돼요.
```

오류가 났지만 `except` 로 처리한 뒤, `finally` 의 마무리 메시지까지 실행됐어요. 오류가 없었어도 `finally` 는 똑같이 실행됐을 거예요.

## 12.4 실전: 안전한 나눗셈 함수

9장에서 배운 함수와 예외 처리를 합쳐, 0으로 나뉘도 죽지 않는 '안전한 나눗셈' 함수를 만들어 볼게요.

#### 파이썬 코드

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "나눗셈 불가"

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

#### 실행 결과

```
5.0
나눗셈 불가
```

정상적인 `safe_divide(10, 2)` 는 5.0을 돌려주고, 0으로 나누는 `safe_divide(10, 0)` 는 오류 대신 '나눗셈 불가'를 돌려줘요. 프로그램이 멈추지 않고 계속 흘러가죠. 이렇게 예외 처리는 프로그램을 튼튼하게 만들어요.

## 12.5 에러 메시지 자세히 보기

`except 오류종류 as e:` 형태로 쓰면 실제 오류 메시지를 `e` 에 담아 볼 수 있어요. 무엇이 문제였는지 자세히 알 수 있죠.

#### 파이썬 코드

```
try:
    lst = [1, 2, 3]
    print(lst[10])    # 없는 번호 → IndexError
except IndexError as e:
    print("IndexError:", e)
```

#### 실행 결과

```
IndexError: list index out of range
```

`lst`에는 값이 3개(0~2번)뿐인데 10번을 꺼내려 해서 `IndexError`가 났어요. 메시지 `list index out of range`는 '리스트 범위를 벗어난 번호'라는 뜻이에요. 메시지를 읽으면 무엇이 잘못됐는지 힌트를 얻을 수 있어요.



#### 예외 처리, 이렇게 기억하세요

- `try` : 오류가 날 수 있는 코드를 넣는 곳
- `except` : 오류가 났을 때 대신 할 일
- `finally` : 오류와 상관없이 꼭 할 일

사용자 입력이나 파일처럼 '무슨 값이 올지 모르는 상황'에서 특히 유용해요.



#### 연습문제

1. 사용자에게 숫자를 입력받아 정수로 바꾸되, 숫자가 아니면 `"숫자를 입력해 주세요!"`라고 안내하도록 `try/except`로 감싸 보세요.
2. 리스트 `[10, 20, 30]`에서 5번 값을 꺼내려 하면 어떤 에러가 날까요? `try/except`로 잡아 메시지를 출력해 보세요.
3. `finally`를 이용해, 오류가 나든 안 나든 마지막에 `"검사 완료"`가 출력되게 만들어 보세요.

#### 해답 · 힌트

① `except ValueError:`로 잡으면 돼요. ② `IndexError` — 없는 번호(범위 초과)라서 나요. `except IndexError as e:`로 잡아 `e`를 출력해 보세요.

# 미니 프로젝트로 정리

드디어 마지막 장이에요! 지금까지 배운 변수·조건문·반복문·함수·자료구조·예외 처리를 모두 동원해서, 진짜 '작동하는 프로그램' 세 개를 만들어 볼 거예요. 개념을 따로따로 배울 때와 다르게, 이것들이 어떻게 어우러지는지 느껴 보세요. 코드는 한 줄씩 설명하니 겁먹지 말고 따라오세요.

## 이 장에서 배울 것

- 미니 프로젝트 1: 간단 계산기
- 미니 프로젝트 2: 할 일 목록(To-Do)
- 미니 프로젝트 3: 숫자 맞추기 게임
- 배운 것을 하나로 엮는 경험

## 13.1 프로젝트 1 — 간단 계산기

두 숫자와 연산자를 받아 계산해 주는 계산기예요. **함수**(9장)로 계산 부분을 묶고, **조건문**(6장)으로 연산자를 구분하고, 0으로 나누는 경우까지 **대비**했어요.

● ● ● 파이썬 코드

```
def calculate(a, op, b):
    if op == "+":
        return a + b
    elif op == "-":
        return a - b
    elif op == "*":
        return a * b
    elif op == "/":
        if b == 0:
            return "0으로 나눌 수 없어요"
        return a / b
    else:
        return "알 수 없는 연산자예요"

# 사용자 입력 받기
a = int(input("첫 번째 숫자: "))
op = input("연산자(+ - * /): ")
b = int(input("두 번째 숫자: "))

print(f"{a} {op} {b} = {calculate(a, op, b)}")
```



12, +, 5 를 입력했다고 하면 이렇게 나와요.

실행 결과

```
12 + 5 = 17
```

20, /, 4 를 넣으면  $20 / 4 = 5.0$ , 0으로 나누면( $7 / 0$ ) 오류 대신 안내가 나와요.

실행 결과

```
7 / 0 = 0으로 나눌 수 없어요
```

### ▶ 코드 한 줄씩 뜯어보기

- `def calculate(a, op, b):` — 세 값(두 숫자와 연산자)을 받는 함수를 정의해요.
- `if op == "+": ... elif ...` — 연산자에 따라 다른 계산을 골라 `return` 해요.
- `if b == 0:` — 나눗셈일 때 0으로 나누는지 미리 검사해 오류를 예방해요.
- `int(input(...))` — 입력을 숫자로 바꿔요(5장). 연산자는 글자라 그대로 받아요.
- `f"{a} {op} {b} = {calculate(a, op, b)}"` — f-string(4장)으로 결과를 예쁘게 조립해요.



연산자 판별을 `if/elif` 로 했지만, 딕셔너리(8장)를 쓰면 더 짧게 만들 수도 있어요. 지금은 배운 것만으로 완성하는 성취감이 더 중요하니, 익숙한 방법으로 만든 거예요. 여러 방법으로 같은 문제를 풀 수 있다는 것도 프로그래밍의 매력입니다.

## 13.2 프로젝트 2 — 할 일 목록(To-Do)

할 일을 추가하고, 목록을 보고, 완료 처리(삭제)하는 프로그램이에요. [리스트](#)(8장)에 할 일을 담고, [while 반복문](#)(7장)으로 메뉴를 계속 보여 주고, [함수](#)로 목록 출력을 묶었어요.

```

todos = [] # 할 일을 담은 리스트

def show_todos(todos):
    if not todos:
        print("할 일이 없어요!")
    else:
        for i, todo in enumerate(todos, start=1):
            print(f"{i}. {todo}")

while True:
    print("\n[1]추가  [2]보기  [3]완료  [4]종료")
    menu = input("선택: ")

    if menu == "1":
        item = input("할 일: ")
        todos.append(item)
        print("추가했어요!")
    elif menu == "2":
        show_todos(todos)
    elif menu == "3":
        done = input("완료한 할 일: ")
        if done in todos:
            todos.remove(done)
            print("완료 처리했어요!")
        else:
            print("그런 할 일이 없어요.")
    elif menu == "4":
        print("안녕히 가세요!")
        break
    else:
        print("1~4 중에 골라 주세요.")

```

추가와 보기를 하면 이런 흐름이에요(핵심 부분만 발췌).

#### 동작 예시

```
=== 현재 할 일 ===
1. 파이썬 공부하기
2. 운동하기
3. 책 읽기
=== '운동하기' 완료! ===
1. 파이썬 공부하기
2. 책 읽기
```

#### ▶ 새로 등장한 기술

- `while True:` — 일부러 무한 반복을 만들고, 종료(4번)를 고르면 `break` 로 빠져나와요. 메뉴 프로그램의 흔한 패턴이예요.
- `enumerate(todos, start=1)` — 리스트를 돌 때 `번호도 함께` 매겨 줘요. 그래서 '1. 2. 3.'처럼 번호를 붙일 수 있어요.
- `if not todos:` — 리스트가 `비었는지` 확인해요. 빈 리스트는 거짓으로 취급돼요.
- `if done in todos:` — 그 값이 리스트에 `있는지` 먼저 확인하고 삭제해요(12장의 예방적 사고와 같은 맥락).



#### **while True**에는 반드시 탈출구를!

`while True` 는 조건이 항상 참이라 `영원히` 돌아요. 안에 반드시 `break` (종료 조건)를 넣어야 프로그램을 끝낼 수 있어요. 7장에서 배운 무한 루프 주의사항, 기억나죠?

## 13.3 프로젝트 3 — 숫자 맞추기 게임

컴퓨터가 정한 숫자를 사용자가 맞추는 게임이에요. `random`(10장)으로 정답을 무작위로 정하고, **반복문**으로 여러 번 기회를 주고, **조건문**으로 '더 크다/작다' 힌트를 줘요. 지금까지의 총정리예요!

파이썬 코드

```
import random

answer = random.randint(1, 20)  # 1~20 중 정답을 무작위로
count = 0

print("1부터 20 사이 숫자를 맞춰 보세요!")

while True:
    guess = int(input("숫자 입력: "))
    count += 1
    if guess < answer:
        print("더 큰 수예요!")
    elif guess > answer:
        print("더 작은 수예요!")
    else:
        print(f"정답! {count}번 만에 맞췄어요! 축하해요!")
        break
```

정답이 5였고, 사용자가 10 → 3 → 5 순으로 입력했다고 하면 이렇게 흘러가요.

실행 예시 (정답=5, 입력: 10 → 3 → 5)

```
1부터 20 사이 숫자를 맞춰 보세요!
더 작은 수예요!
더 큰 수예요!
정답! 3번 만에 맞췄어요! 축하해요!
```

### ▶ 이 게임에 녹아든 것들

- `random.randint(1, 20)` — 정답을 무작위로 정해 매번 다른 게임이 돼요(10장).
- `while True:` + `break` — 맞힐 때까지 계속 물어보다가, 맞으면 탈출(7·13장).
- `count += 1` — 시도 횟수를 누적해 세요(3·7장의 누적 패턴).
- `if/elif/else` — 입력값과 정답을 비교해 힌트를 줘요(6장).



정답이 무작위라 실행할 때마다 게임이 달라져요. 이 책에선 설명을 위해 정답을 5로 고정한 예시를 보여 줬지만, 여러분이 직접 실행하면 매번 새로운 숫자가 나와요. 친구와 번갈아 도전해 보세요!

## 13.4 축하합니다, 1권을 완주했어요!

정말 대단해요. 여러분은 이제 파이썬으로 변수에 값을 담고, 조건에 따라 판단하고, 반복으로 일을 자동화하고, 함수로 코드를 정리하고, 여러 값을 자료구조로 다루고, 파일에 기록하고, 오류에 대처할 줄 알아요. 이 세 프로젝트가 그 증거예요. 처음

`print("Hello, World!")` 를 찍던 게 엇그제 같은데, 어느새 작동하는 프로그램을 손수 만들었네요.



#### 가장 좋은 공부는 '직접 만들어 보기'예요

이 책의 예제를 그대로 따라 친 다음, 조금씩 바꿔 보세요. 계산기에 거듭제곱을 추가하거나, 할 일에 '수정' 기능을 넣거나, 숫자 범위를 100까지 넓혀 보는 거죠. 직접 고치고 오류를 만나며 배우는 게 진짜 실력이 돼요. 오류는 여러분의 스승이예요.

파이썬이 정말 쉽고 재미있게 느껴졌길 바라요. 여기서 다진 기초는 앞으로 무엇을 배우든 든든한 밑거름이 될 거예요. 더 넓은 세계 — 나만의 도구를 만드는 클래스, 남이 만든 강력한 외부 라이브러리, 그리고 웹·데이터·자동화 같은 실전 응용 — 은 다음 편에서 이어서 함께 걸어가요. 여기까지 온 당신, 이미 훌륭한 파이썬 입문자예요. 정말 수고 많았어요!